Call by Value and Call by Reference (Full Notes)

When a function is called in C, arguments can be passed in **two ways**:

1. Call by Value
2. Call by Reference

◆ 1. Call by Value

Definition:

In **Call by Value**, a **copy of the variable** is passed to the function.

Changes made inside the function **DO NOT affect** the original variable.

◆ How It Works:

- Actual parameters → copied into → formal parameters
- Function works on copied data
- Original values remain unchanged

◆ Example Program (Call by Value)

```
#include <stdio.h>

void change(int x) {
    x = 50;
    printf("Inside function: %d\n", x);
}

int main() {
    int a = 10;

    change(a);

    printf("Outside function: %d\n", a);

    return 0;
}
```

◆ Output:

```
Inside function: 50
Outside function: 10
```

Explanation:

- `a` is copied into `x`
- Changing `x` does not change `a`

◆ Advantages of Call by Value

✓ Original data is safe
✓ No accidental modification
✓ Easy to understand

◆ Disadvantages

✕ Cannot modify original variables
✕ Extra memory used for copy

◆ 2. Call by Reference

Definition:

In **Call by Reference**, the **address of the variable** is passed to the function.

Changes made inside the function **affect the original variable**.

In C, this is done using **pointers**.

◆ How It Works:

- Address of variable is passed
- Function receives pointer
- Changes directly modify original memory location

Program 1: Addition of Two Numbers (Call by Reference)

```
#include <stdio.h>

void add(int *x, int *y, int *result) {
    *result = *x + *y;
}

int main() {
    int a = 10, b = 20, sum;

    add(&a, &b, &sum);

    printf("Sum = %d\n", sum);

    return 0;
}
```

Explanation:

- `&a` → address of a
- `*x` → value at that address
- Original variables are accessed using pointers

Program 2: Swap Two Numbers (Call by Reference)

```
#include <stdio.h>

void swap(int *x, int *y) {
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;

    printf("Before Swap: a = %d, b = %d\n", a, b);

    swap(&a, &b);

    printf("After Swap: a = %d, b = %d\n", a, b);

    return 0;
}
```

◆ **Output:**

```
Before Swap: a = 5, b = 10
After Swap: a = 10, b = 5
```

🔎 Explanation:

- Addresses of `a` and `b` are passed
- Values are swapped at original memory location
- Hence original variables change

Difference Between Call by Value and Call by Reference

Call by Value	Call by Reference
Copy of variable passed	Address of variable passed
Original value not changed	Original value changed
Uses normal variables	Uses pointers
Safer	Faster for large data

Memory Diagram Understanding

Call by Value

```
a = 10

change(a)

x = 10 (copy)
```

Original `a` remains 10.

Call by Reference

```
a = 10
Address of a → 1000

swap(&a)

Pointer x → 1000
```

Function modifies value at address 1000.

◆ When to Use What?

Use Call by Value:

- When you don’t want original data changed
- Small data types

Use Call by Reference:

- When you need to modify original variables
- Large structures/arrays
- To return multiple values